UNIVERSIDADE FEDERAL DE SERGIPE
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIA
DEPARTAMENTO DE COMPUTAÇÃO

THYFANNY DA CRUZ SILVA
FLAVIO DA SILVA INACIO DOS SANTOS

PROJETO MAPTHESOUL

RELATÓRIO TÉCNICO SOBRE ÁRVORES DE DECISÃO

SÃO CRISTÓVÃO - SE
2026

SUMÁRIO

1. INTRODUÇÃO.....3

1.1. DEFINIÇÃO DO PROBLEMA.....3

2. ÁRVORE DE DECISÃO.....3

2.1 SELEÇÃO E ENGENHARIA DE ATRIBUTOS.....4

2.2. IMPLEMENTAÇÃO E CORRESPONDÊNCIA TEÓRICA.....4

3. INTERFACE GRÁFICA.....7

4. AVALIAÇÃO DO MODELO.....9

5. CONCLUSÃO.....11

1. INTRODUÇÃO

O MapTheSoul é um ecossistema computacional baseado em web, desenvolvido para atuar como um sistema de suporte à decisão no domínio cinematográfico. A aplicação utiliza técnicas de Aprendizado de Máquina (*Machine Learning*) para filtrar e recomendar obras audiovisuais a partir de quatro eixos fundamentais de preferência do usuário: gênero, duração, período histórico e limiar de aceitação crítica (nota mínima no IMDB).

1.1. DEFINIÇÃO DO PROBLEMA

O crescimento das plataformas de streaming trouxe um efeito curioso: embora o usuário tenha acesso a um catálogo cada vez maior, a quantidade de opções pode dificultar a escolha e reduzir a satisfação com a decisão tomada. Esse fenômeno, conhecido como paradoxo da escolha, pode ser analisado sob a perspectiva da Ciência da Computação como um problema de classificação.

Nesse contexto, o catálogo de filmes é tratado como um espaço de busca com múltiplas características, no qual cada item deve ser enquadrado em uma de duas categorias: recomendado ou não recomendado. O objetivo do modelo é identificar padrões nos dados que permitam prever, de forma consistente, a classe mais adequada para cada filme.

Para essa tarefa, foi adotado o algoritmo de árvores de decisão, que organiza o processo de tomada de decisão em uma estrutura hierárquica de regras. Durante o treinamento, o modelo realiza divisões sucessivas no conjunto de dados com base nos atributos mais relevantes, formando ramos que conduzem a nós finais responsáveis pela predição. Esse mecanismo permite representar decisões complexas de maneira interpretável e alinhada ao comportamento observado nos dados.

2. ÁRVORE DE DECISÃO

Árvores de decisão são algoritmos de aprendizado supervisionado baseados na divisão progressiva do conjunto de dados em subconjuntos mais homogêneos. O modelo assume a forma de uma estrutura hierárquica composta por três elementos principais: os nós internos, que realizam testes sobre atributos; os ramos, que representam os possíveis resultados desses testes; e os nós folha, responsáveis

pela classificação final das instâncias, neste caso como “recomendado” ou “não recomendado”.

A escolha desse algoritmo se justifica principalmente por duas características. A primeira é a interpretabilidade, já que a árvore permite acompanhar o caminho lógico percorrido até a predição, tornando o processo de recomendação transparente. A segunda é a capacidade de lidar com dados heterogêneos, combinando variáveis numéricas e categóricas sem a necessidade de transformações complexas.

2.1 SELEÇÃO E ENGENHARIA DE ATRIBUTOS

O sucesso do treinamento depende da seleção de variáveis que capturem a "essência" do que torna um filme recomendável. O modelo utiliza o seguinte vetor de características (*feature vector*):

- Genre_encoded: Representação numérica das categorias de filme. Devido à natureza multirrótulo dos filmes, aplicou-se a técnica de Explode, transformando listas de gêneros em entradas individuais, seguidas por um Label Encoding.
- Runtime: Variável contínua representando a duração em minutos.
- Meta_score: Indicador numérico de qualidade técnica atribuído pela crítica especializada.
- No_of_Votes: Métrica de popularidade que reflete o engajamento do público global.
- Released_Year: Variável temporal convertida para formato numérico para permitir a identificação de padrões sazonais ou geracionais.

2.2. IMPLEMENTAÇÃO E CORRESPONDÊNCIA TEÓRICA

A base lógica do treinamento do modelo reside no algoritmo LEARN-DECISION-TREE, detalhado na obra Artificial Intelligence: A Modern Approach (Russell & Norvig). Este algoritmo, fundamentado no clássico ID3, utiliza uma abordagem recursiva top-down para a construção da árvore.

O processo exige três parâmetros fundamentais para sua execução: examples (conjunto de treinamento), attributes (vetor de características) e parent_examples (dados do nó antecessor, utilizados para lidar com casos de ausência de dados).

O método train da classe MapTheSoulTrainer (Figura 1) é o responsável por orquestrar essa lógica. Embora utilize a biblioteca Scikit-Learn para ganho de performance, a sequência de operações segue o rigor do algoritmo clássico.

Figura 1 - Implementação do método de treinamento da Árvore de Decisão

```python
def train(self, df: pd.DataFrame) -> Tuple[pd.DataFrame, pd.Series]:
    """Trains the Decision Tree model and returns the test set for evaluation."""
    x = df[self._features]
    y = df[self._target]

    x_train, x_test, y_train, y_test = train_test_split(
        x, y, test_size=0.2, random_state=42, stratify=y
    )

    # Using max_depth to prevent overfitting
    self._model = DecisionTreeClassifier(max_depth=5, criterion="gini", random_state=42)
    self._model.fit(x_train, y_train)

    return x_test, y_test
```

A Figura 2 detalha como o modelo treinado é utilizado para filtrar o catálogo e gerar sugestões personalizadas.

Figura 2 - Implementação do método de previsão da Árvore de Decisão

```python
def recomendation(genero: str, duracao, decada, nota_minima, model, encoder, maps, df):
    """Filter and rank movies based on user preferences"""
    df_filtrado = df[df["Genre"].apply(lambda x: genero in [g.strip() for g in x.split(",")])].copy()

    min_dur, max_dur = maps["duracao"][duracao]
    df_filtrado = df_filtrado[(df_filtrado["Runtime"] >= min_dur) & (df_filtrado["Runtime"] <= max_dur)]

    if maps["decada"][decada] is not None:
        min_ano, max_ano = maps["decada"][decada]
        df_filtrado = df_filtrado[(df_filtrado["Released_Year"] >= min_ano) & (df_filtrado["Released_Year"] <= max_ano)]

    df_filtrado = df_filtrado[df_filtrado["IMDB_Rating"] >= nota_minima]

    if df_filtrado.empty:
        return pd.DataFrame()

    df_filtrado["Genre_encoded"] = encoder.transform([genero] * len(df_filtrado))
    features = ["Genre_encoded", "Runtime", "Meta_score", "No_of_Votes"]

    df_filtrado["Recomendado"] = model.predict(df_filtrado[features])

    return df_filtrado[df_filtrado["Recomendado"] == 1][
        ["Series_Title", "Genre", "IMDB_Rating", "Runtime", "Released_Year"]
    ].sample(frac=1).head(10)
```

Figura 3 - Pseudocódigo de Referência (ID3)

```
LEARN-DECISION-TREE(examples, attributes, parent_examples):
        if examples is empty:
            return PLURALITY-VALUE(parent_examples)
        else if all examples have the same classification:
            return the classification
        else if attributes is empty:
            return PLURALITY-VALUE(examples)
        else:
            A <- argmax IMPORTANCE(a, examples)
            tree <- a new decision tree with root test A
            for each value v of A:
                exs <- {e : e in examples and e.A = v}
                subtree <- LEARN-DECISION-TREE(exs, attributes - A, examples)
                add branch to tree with label (A = v) and subtree
            return tree
```

Abaixo, está contido os relacionamentos dos componentes teóricos clássicos com as classes e métodos envolvidos neste projeto:

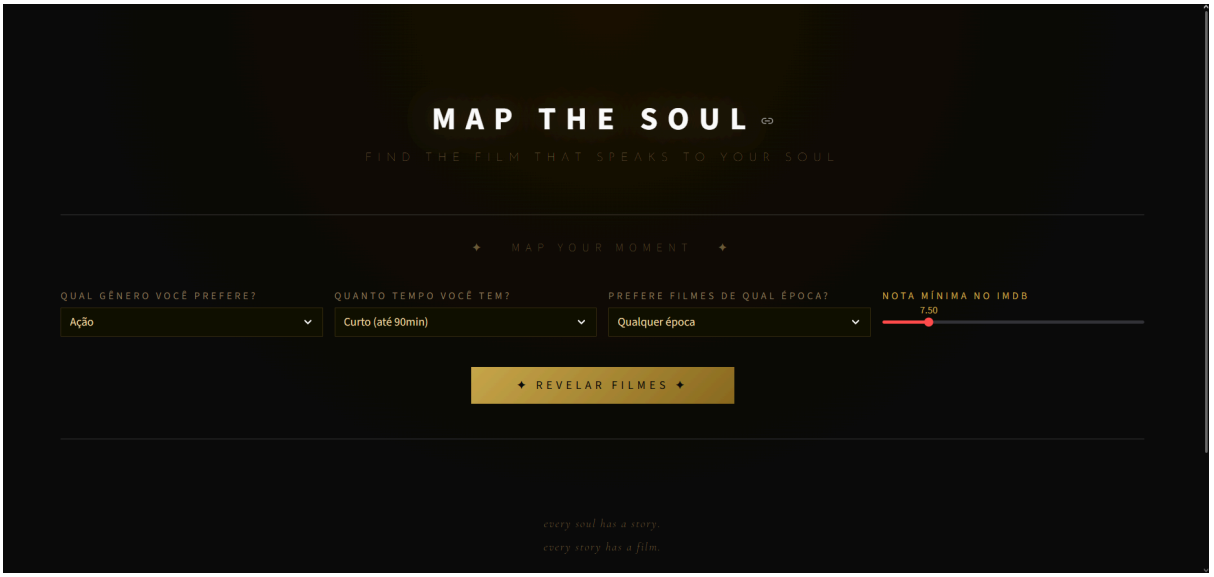
Tabela 1 - Tabela de Mapeamento Lógico

Elemento no pseudocódigo (Russel & Norvig)	Implementação no Projeto (Python)	Descrição Técnica
examples	x_train, y_train	Conjunto de filmes com seus atributos e rótulos usados para treinar o modelo.
attributes	features = [“Genre_encoded”, “Runtime”, “Meta_score”, “No_of_Votes”]	Características dos filmes disponíveis para a árvore fazer divisões.
IMPORTANCE(a, examples)	criterion=”gini”	Calcula a impureza de cada atributo e seleciona o que melhor separa as classes
Condições de parada	max_depth=5, random_state=42	Limita a profundidade da árvore, evitando o overfitting.
tree <- new decision tree / model.fit()	model.fit(X_train, y_train)	Executa a construção recursiva da árvore, dividindo os dados nó a nó.
PLURALITY-VALUE(examples)	Classe majoritária na folha (interno ao sklearn)	Quando não há mais atributos ou exemplos, retorna a classe mais comum
return classification	model.predict(df_filtrado[features])	Percorre a árvore até a folha e retorna Recomendado(1) ou Não Recomendado(0)

3. INTERFACE GRÁFICA

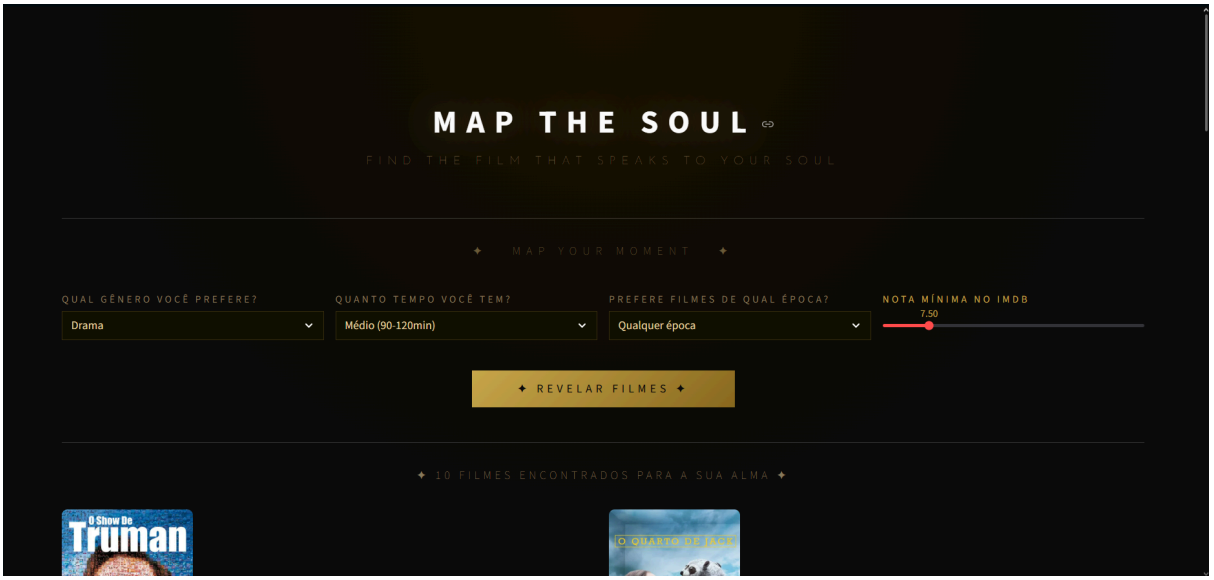
A interface foi feita de forma a ser simples e de fácil entendimento para o usuário, com poucas informações na tela, e permitindo que tudo seja feito em uma única tela.

Figura 4 - Tela principal da aplicação



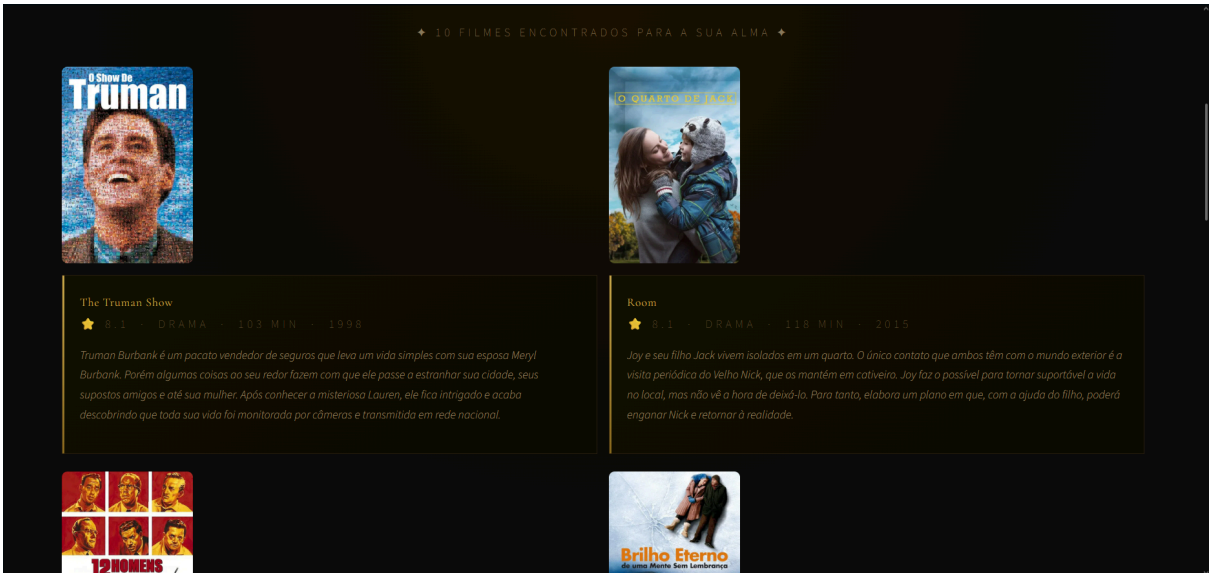
No meio da tela, ficam os campos para a seleção do usuário, que funcionam em forma de drop down e slidebar para a nota mínima no IMDB.

Figura 5 - Tela principal da aplicação



Nesse exemplo, foram selecionados filmes de drama, com duração média (entre 90 a 120 minutos), lançados em qualquer época, com nota mínima de 7.5 no IMDB.

Figura 6 - Filmes recomendados com base nas preferências selecionadas



4. AVALIAÇÃO DO MODELO

A performance do modelo foi analisada a partir de um conjunto de teste separado, composto por 20% dos dados originais. Essa divisão permitiu avaliar como o classificador se comporta diante de exemplos não vistos durante o treinamento, oferecendo uma estimativa mais realista de sua capacidade de generalização. Os resultados obtidos são apresentados na Tabela 2 e discutidos na sequência.

O modelo foi avaliado no conjunto de teste (20% dos dados) com os seguintes resultados:

Tabela 2 – Métricas de Desempenho do Classificador

Métrica	Valor
Acurácia	79.25%
Verdadeiros Negativos	229
Falsos Positivos	23
Falsos Negativos	66
Verdadeiros Positivos	111

A acurácia de 79,25% indica que o modelo apresenta um desempenho preditivo consistente. Ainda assim, em sistemas de recomendação, a simples taxa de acerto não é suficiente, sendo importante observar a natureza dos erros cometidos.

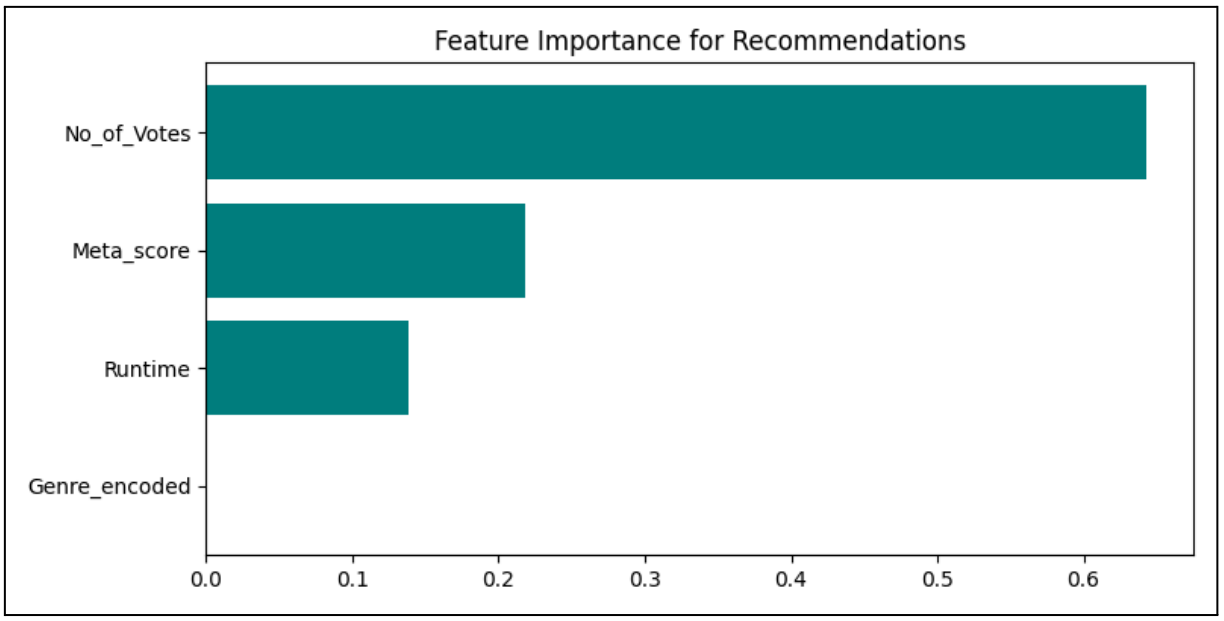
O número reduzido de falsos positivos (23) reflete uma boa precisão, de 82,83%. Isso significa que, quando o MapTheSoul sugere um filme, há grande chance de a recomendação corresponder a obras bem avaliadas (nota maior ou igual a 8,0), diminuindo a probabilidade de frustração do usuário com indicações pouco relevantes.

Por outro lado, a presença de 66 falsos negativos, evidenciada pelo recall, sugere um comportamento mais conservador do modelo. Em vez de arriscar recomendações inadequadas, o sistema tende a deixar de indicar alguns filmes de qualidade, priorizando a confiabilidade das sugestões apresentadas.

Além disso, o elevado número de verdadeiros negativos (229) mostra que o classificador é eficiente ao descartar obras que não atendem aos critérios estabelecidos, contribuindo para reduzir o conjunto de opções e tornar o processo de escolha mais direcionado.

Após a consolidação do modelo, mensurou-se a relevância de cada variável para a estrutura de decisão do classificador. A Figura 7 detalha a importância das quatro métricas preditoras, destacando como o peso estatístico de cada uma influenciou diretamente a partição dos nós e a hierarquia da árvore.

Figura 7 – Importância das Variáveis na Classificação de Recomendações



O gráfico da figura 7 mostra que o número de votos é o atributo com maior influência no processo de recomendação, apresentando peso próximo de 0,6. Esse resultado

sugere que a popularidade e o nível de engajamento do público funcionam como os principais indicadores de qualidade dentro do conjunto de dados analisados.

Em seguida, o *meta score* aparece como o segundo fator mais relevante, com peso em torno de 0,2. Nesse caso, a avaliação da crítica especializada atua como um complemento importante, ajudando a validar tecnicamente as decisões da árvore.

Já a duração do filme e o gênero codificado apresentam menor impacto relativo. Embora participem do refinamento das decisões ao longo da estrutura do modelo, esses atributos não exercem papel central nas primeiras divisões realizadas pela árvore.

5. CONCLUSÃO

O MapTheSoul demonstra como algoritmos clássicos de aprendizado de máquina, como a Árvore de Decisão, podem ser aplicados de forma eficaz a problemas cotidianos como a recomendação de filmes.

O projeto entrega uma aplicação funcional, com interface elegante e intuitiva, capaz de recomendar filmes personalizados com base nas preferências do usuário, demonstrando na prática o potencial dos algoritmos de Inteligência Artificial para resolver problemas reais.